Canvas Line Sender
코드 블록 중심 초상세 해설서

이벤트 처리 블록, 삭제/초기화, 점 재생성, JSON 전송, 좌표 변환을
코드 조각과 함께 중학생도 이해할 수 있게 풀어쓴 설명서

이 문서는 이전 설명서보다 더 자세하게, 실제 코드 블록을 먼저 보여 준 뒤 바로 아래에서 줄 흐름을 풀어 설명하는 방식으로 구성했다. 그래서 "이 줄이 왜 필요한지", "이 조건이 참일 때 무슨 일이 일어나는지", "다음 이벤트와 어떻게 연결되는지"를 코드 가까이에서 이해할 수 있다.

읽는 방법은 간단하다. 각 절에서 먼저 코드 블록을 보고, 그다음 설명을 따라가면 된다. 특히 _onPointerDown, _onPointerMove, _onPointerUp 같은 이벤트 처리 함수는 마우스를 누르고, 움직이고, 떼는 순서를 그대로 따라 설명한다.

1. 먼저 큰 흐름부터: 이 앱에서 이벤트가 어떻게 이어지는가

- 사용자가 마우스를 누르면 _onPointerDown 이 실행된다. 이때 "그리기를 시작할 준비"를 한다.
- 마우스를 움직이면 _onPointerMove 가 반복 실행된다. 이때 선 끝점이나 pen 경로가 계속 갱신된다.
- 마우스를 떼면 _onPointerUp 이 실행된다. 이때 임시로 그리고 있던 내용을 하나의 stroke 로 확정한다.
- 그 과정에서 _rebuildPoints 가 자주 호출된다. 이 함수가 현재 그리기 상태를 실제 점 목록(points)으로 다시 만드는 핵심 함수다.
- 완성된 점들은 _buildPayload 에서 JSON 형식으로 바뀌고, _sendJson 이 소켓으로 서버에 전송한다.

2. Dart 와 Flutter 문법을 먼저 아주 쉽게

- void 함수이름() 은 "이 함수는 어떤 값을 돌려주지 않는다"는 뜻이다.
- setState(() { ... }) 는 "상태가 바뀌었으니 화면을 다시 그려 달라"는 명령이다.
- if (...) { ... } else { ... } 는 조건 분기다. 상황에 따라 다른 코드를 실행한다.
- final 은 한 번 정한 값을 다시 바꾸지 않겠다는 뜻이다.
- Offset 은 x, y 좌표를 담는 Flutter 클래스다. 점 하나라고 생각하면 된다.
- List<Offset> 은 점 여러 개를 담는 리스트다.
- ? 는 null 이 될 수도 있다는 뜻이고, ! 는 "지금은 null 이 아니라고 확신한다"는 뜻이다.
- event.localPosition 은 사용자가 현재 위젯 안에서 클릭한 좌표다.

3. 핵심 이벤트 1: _onPointerDown

사용자가 마우스를 누르는 순간 실행된다. 이 함수는 "지금부터 어떤 방식으로 그리기를 시작할지"를 정한다. line 모드인지, pen 모드인지, 이어그리기(Continue)가 켜져 있는지에 따라 내부 동작이 달라진다.

```
setState(() {
  _isDrawing = true;

  if (_mode == DrawMode.line) {
    final startPos = _clampToFrame(event.localPosition);
    if (_appendPen && _strokes.isNotEmpty) {
      _appendStrokeIndex = _strokes.length - 1;
      final last =
          _strokes.last.path.isNotEmpty ? _strokes.last.path.last : null;
      _p0 = last ?? startPos;
      _p1 = startPos;
    } else {
      _appendStrokeIndex = null;
      _p0 = startPos;
      _p1 = startPos;
    }
  } else {
    if (_mode == DrawMode.pen && _appendPen && _strokes.isNotEmpty) {
      _appendStrokeIndex = _strokes.length - 1;
      final last =
          _strokes.last.path.isNotEmpty ? _strokes.last.path.last : null;
      if (last != null) {
        _rawPoints = <Offset>[last, _clampToFrame(event.localPosition)];
        _p0 = last;
      } else {
        final pos = _clampToFrame(event.localPosition);
        _rawPoints = <Offset>[pos];
        _p0 = pos;
      }
    } else {
      _appendStrokeIndex = null;
      final pos = _clampToFrame(event.localPosition);
      _rawPoints = <Offset>[pos];
      _p0 = pos;
    }
    _p1 = _clampToFrame(event.localPosition);
  }

  _rebuildPoints();
});
```

- _isDrawing = true 는 "이제 사용자가 실제로 그리기 시작했다"는 표시다. 이후 move 이벤트에서 이 값을 보고 계속 갱신할지 말지를 결정한다.
- if (_mode == DrawMode.line) 는 직선 모드일 때와 나머지 모드(pen, circle, rect)를 나누는 가장 큰 분기다. 직선은 시작점과 끝점만 있으면 되지만, pen 은 지나간 점들을 리스트로 쌓아야 하므로 처리 방식이 다르다.

- _clampToFrame(event.localPosition) 는 클릭 좌표를 프레임 내부로 제한한다. 사용자가 약간 바깥을 눌러도 계산이 망가지지 않게 하는 안전장치다.
- _appendPen && _strokes.isNotEmpty 가 참이면 Continue 기능이 켜져 있고 기존 선도 있다는 뜻이다. 그래서 새 선을 완전히 새로 시작하지 않고 마지막 stroke 뒤에 붙일 준비를 한다.
- _appendStrokeIndex = _strokes.length - 1 은 "마지막 stroke 에 이어붙일 거야"라는 기록이다.
- final last = ... ? ... : null 은 마지막 stroke 의 끝점을 꺼낸다. 삼항 연산자(조건 ? 참일 때 값 : 거짓일 때 값)를 사용한 Dart 문법 예시다.
- _p0 = last ?? startPos 는 last 가 있으면 그 점을 시작점으로 쓰고, 없으면 지금 클릭한 점(startPos)을 시작점으로 쓰라는 뜻이다. ?? 는 null 병합 연산자다.
- pen 모드에서 _rawPoints = <Offset>[last, 현재점] 처럼 리스트를 만드는 이유는 펜 경로가 점 여러 개의 연속이기 때문이다. 처음부터 2 개 점을 넣어 두면 이어그리기가 더 자연스럽다.
- 마지막의 _rebuildPoints() 는 이 함수의 핵심 마무리다. 지금까지 설정한 _p0, _p1, _rawPoints 를 바탕으로 실제 샘플 점 목록을 다시 만든다.

4. 핵심 이벤트 2: _onPointerMove

마우스를 움직이는 동안 반복 실행된다. 현재 상태가 이동 중인지, 점 편집 중인지, 박스 선택 중인지, 일반 그리기 중인지에 따라 서로 다른 동작을 한다.

```
void _onPointerMove(PointerMoveEvent event) {
 if (_isMoving) {
  _updateMove(event.localPosition);
  return;
 }
 if (_isEditingPoint) {
  _updateEditPoint(event.localPosition);
  return;
 }
 if (_isSelecting) {
  setState(() {
   _selectRect = Rect.fromPoints(_selectStart!, event.localPosition);
  });
  return;
 }
 if (!_isDrawing) return;
 setState(() {
  if (_mode == DrawMode.line) {
   _p1 = _clampToFrame(event.localPosition);
  } else {
   final pos = _clampToFrame(event.localPosition);
   _rawPoints.add(pos);
   _p1 = pos;
  }
  _rebuildPoints();
 });
}
```

- 이 함수의 첫 특징은 “우선순위 분기”다. 이동, 점 편집, 선택 박스가 일반 그리기보다 먼저 검사된다. 그래서 서로 다른 작업이 충돌하지 않는다.
- _isMoving 이 참이면 선 전체를 드래그하는 중이라는 뜻이다. 이때는 _updateMove 만 하고 바로 return 한다.
- _isEditingPoint 가 참이면 Shift+드래그 등으로 샘플 점 하나를 수정하는 중이다. 그래서 점 편집만 수행하고 빠져나온다.
- _isSelecting 이 참이면 드래그 박스를 키우는 중이다. Rect.fromPoints(start, current) 로 사각형을 매 순간 다시 만든다.
- if (!_isDrawing) return 은 굉장히 중요하다. 마우스를 그냥 움직이는 것과 실제로 그리는 중인 것을 구분한다.
- line 모드에서는 끝점 _p1 만 계속 바꾸면 된다. 그러면 시작점 _p0 에서 현재 마우스 위치까지 직선 미리보기가 된다.
- pen 모드에서는 현재 점을 _rawPoints 에 계속 추가한다. 즉 손으로 그린 경로를 점 리스트로 쌓는다.
- 여기서도 마지막에 _rebuildPoints() 가 호출된다. move 할 때마다 점 목록이 다시 계산되기 때문에 화면에 부드럽게 업데이트된다.

5. 핵심 이벤트 3: _onPointerUp

마우스를 떼는 순간 실행된다. 지금까지 임시 상태로 그리고 있던 것을 진짜 stroke 로 저장하는 단계다.

```
void _onPointerUp(PointerUpEvent event) {
  if (_isMoving) {
    _endMove();
    return;
  }
  if (_isEditingPoint) {
    _endEditPoint();
    return;
  }
  if (_isSelecting) {
    final rect = _selectRect;
    setState(() {
      _isSelecting = false;
      _selectStart = null;
      _selectRect = null;
    });
    if (rect != null) {
      _selectStrokeByRect(rect);
    }
    return;
  }
  if (!_isDrawing) return;
```

```
setState(() {
  _isDrawing = false;
  if (_mode == DrawMode.line) {
    _p1 = _clampToFrame(event.localPosition);
  } else {
    final pos = _clampToFrame(event.localPosition);
    _rawPoints.add(pos);
    _p1 = pos;
  }

  _rebuildPoints();

  if (_pointsPx.isNotEmpty) {
    final path = _mode == DrawMode.pen
        ? List<Offset>.from(_rawPoints)
        : List<Offset>.from(_pointsPx);
    final samples = List<Offset>.from(_pointsPx);

    if ((_mode == DrawMode.pen || _mode == DrawMode.line) &&
        _appendPen &&
        _appendStrokeIndex != null) {
      final stroke = _strokes[_appendStrokeIndex!];

      if (stroke.path.isNotEmpty &&
          path.isNotEmpty &&
          stroke.path.last == path.first) {
        path.removeAt(0);
      }
      if (stroke.samples.isNotEmpty &&
          samples.isNotEmpty &&
          stroke.samples.last == samples.first) {
        samples.removeAt(0);
      }

      stroke.path.addAll(path);
      stroke.samples.addAll(samples);
      stroke.zValues.addAll(List<double>.filled(samples.length, 0.0));
    } else {
      _strokes.add(
        Stroke(
          path: path,
          samples: samples,
          zValues: List<double>.filled(samples.length, 0.0),
        ),
      );
    }
  }
  _appendStrokeIndex = null;
});
}
```

- 마우스를 떼는 시점은 “이제 그림 하나를 완성할지, 이동/편집/선택을 끝낼지”를 확정하는 순간이다.
- _isDrawing = false 로 바꾸는 이유는 더 이상 그리고 있지 않음을 표시하기 위해서다.
- line 모드면 마지막 위치를 _p1 에 넣고, pen 모드면 마지막 점을 _rawPoints 에 추가한다. 즉 최종 좌표를 한 번 더 확정한다.

- _rebuildPoints() 를 한 번 더 호출하는 이유는 마지막 좌표까지 포함한 완성본 점 목록을 만들기 위해서다.
- path 와 samples 를 따로 만드는 이유가 중요하다. path 는 사용자가 실제로 지나간 경로나 도형 경로에 가깝고, samples 는 일정 간격으로 재샘플링된 점들이다.
- Continue 모드로 이어붙일 때는 맨 앞 점이 이전 stroke 의 마지막 점과 겹칠 수 있다. 그래서 path.removeAt(0), samples.removeAt(0) 로 중복 점을 제거한다.
- stroke.zValues.addAll(List<double>.filled(samples.length, 0.0)) 는 새로 생긴 샘플 점마다 기본 Z 값 0.0 을 넣는 코드다.
- 이어그리기가 아니면 완전히 새로운 Stroke 객체를 만들어 _strokes 리스트에 추가한다. 이 순간부터 한 개의 완성된 선으로 관리된다.

6. 삭제 함수: _deleteSelectedOrLast

사용자가 Delete 버튼을 눌렀을 때 실행된다. 선택된 stroke 가 있으면 그걸 지우고, 없으면 가장 마지막 stroke 를 지운다.

```
void _deleteSelectedOrLast() {
  setState(() {
    if (_strokes.isEmpty) return;
    if (_selectedStrokeIndex != null &&
        _selectedStrokeIndex! >= 0 &&
        _selectedStrokeIndex! < _strokes.length) {
      _strokes.removeAt(_selectedStrokeIndex!);
      if (_strokes.isEmpty) {
        _selectedStrokeIndex = null;
      } else if (_selectedStrokeIndex! >= _strokes.length) {
        _selectedStrokeIndex = _strokes.length - 1;
      }
    } else {
      _strokes.removeLast();
    }
    _clearCurrentDrawing();
  });
}
```

- if (_strokes.isEmpty) return 은 지울 선이 하나도 없으면 바로 끝내는 보호 코드다.
- _selectedStrokeIndex != null ... < _strokes.length 조건은 “선택 인덱스가 실제로 유효한가?”를 점검한다.
- _strokes.removeAt(_selectedStrokeIndex!) 는 선택한 선을 특정 위치에서 삭제한다.
- 삭제 후 _strokes 가 비면 더 이상 선택할 대상이 없으므로 _selectedStrokeIndex = null 로 바꾼다.
- 삭제 전에는 마지막 선이 선택되어 있었는데 삭제 후 리스트 길이가 줄어들면, 기존 인덱스가 범위를 벗어날 수 있다. 그래서 마지막 유효 인덱스로 다시 맞춘다.
- 선택된 대상이 없다면 removeLast() 로 최근에 그린 선부터 지운다.

- _clearCurrentDrawing() 은 화면에 남아 있을 수 있는 임시 점/미리보기 정보를 함께 비워 주는 정리 단계다.

7. 초기화 함수: _reset 과 _clearCurrentDrawing

둘 다 "지운다"는 느낌이 있지만 범위가 다르다. _reset 은 거의 전체 상태를 초기화하고, _clearCurrentDrawing 은 현재 임시 드로잉 정보만 지운다.

```
void _reset() {
  setState(() {
    _p0 = null;
    _p1 = null;
    _rawPoints = <Offset>[];
    _pointsPx = <Offset>[];
    _pointsImage = <Offset>[];
    _strokes.clear();
    _dragStrokeIndex = null;
    _dragLast = null;
    _isMoving = false;
    _selectedStrokeIndex = null;
    _appendStrokeIndex = null;
    _isSelecting = false;
    _selectStart = null;
    _selectRect = null;
    _pointsMm = <Offset>[];
    _deltasMm = <Offset>[];
    _currentZValues = <double>[];
    _status = 'Reset';
  });
}

void _clearCurrentDrawing() {
  setState(() {
    _p0 = null;
    _p1 = null;
    _rawPoints = <Offset>[];
    _pointsPx = <Offset>[];
    _pointsImage = <Offset>[];
    _pointsMm = <Offset>[];
    _deltasMm = <Offset>[];
    _currentZValues = <double>[];
    _appendStrokeIndex = null;
    _isDrawing = false;
  });
}
```

- _reset 은 전체 캔버스를 새 파일처럼 만드는 함수다. 저장된 stroke 도 전부 지워진다.
- _strokes.clear() 가 들어 있다는 점이 _clearCurrentDrawing 과 가장 큰 차이다.
- _clearCurrentDrawing 은 이미 저장된 stroke 들은 남겨 둔다. 대신 지금 막 그리고 있던 임시 선만 비운다.

- 즉 _reset 은 “전체 초기화”, _clearCurrentDrawing 은 “현재 미리보기/작업 중 상태만 정리”라고 생각하면 된다.

8. 가장 중요한 계산 함수: _rebuildPoints

이 함수는 현재 모드와 입력 상태를 보고 실제 점 리스트를 다시 만든다. 이 앱의 좌표 계산 중심축이라고 볼 수 있다.

```
void _rebuildPoints() {
  if ((_mode == DrawMode.line ||
      _mode == DrawMode.circle ||
      _mode == DrawMode.rect) &&
      (_p0 == null || _p1 == null)) {
    _pointsPx = <Offset>[];
    _pointsImage = <Offset>[];
    _pointsMm = <Offset>[];
    _deltasMm = <Offset>[];
    _currentZValues = <double>[];
    return;
  }

  List<Offset> points;
  List<Offset> pointsImage;

  if (_mode == DrawMode.line) {
    ...
  } else if (_mode == DrawMode.circle) {
    ...
  } else if (_mode == DrawMode.rect) {
    ...
  } else {
    final rawImage =
        _rawPoints.map(_canvasToImageClamped).toList(growable: false);
    pointsImage = _resamplePolyline(rawImage, _stepPx);
    points = _imagePointsToCanvas(pointsImage);
  }

  _pointsPx = points;
  _pointsImage = pointsImage;

  if (pointsImage.isEmpty) {
    _pointsMm = <Offset>[];
    _deltasMm = <Offset>[];
    _currentZValues = <double>[];
    return;
  }

  _pointsMm = _applyTransform(pointsImage, _mmPerPx);
  _deltasMm = _buildDeltas(_pointsMm);
  _currentZValues = _syncZList(_currentZValues, _pointsMm.length);
}
```

- 처음의 guard 절은 “도형 모드인데 시작점/끝점이 아직 없으면 계산할 것이 없다”는 뜻이다.

- points 와 pointsImage 를 따로 두는 이유는 좌표계가 다르기 때문이다. 하나는 캔버스 표시용, 하나는 이미지/프레임 기준 계산용이다.
- pen 모드에서는 _rawPoints 를 이미지 좌표계로 바꾸고, _resamplePolyline 으로 일정 간격(stepPx)으로 다시 샘플링한다.
- 그 뒤 _imagePointsToCanvas 로 화면 표시용 좌표를 다시 만든다. 즉 계산 좌표와 화면 좌표가 서로 왔다 갔다 한다.
- _pointsMm = _applyTransform(pointsImage, _mmPerPx) 는 이미지 좌표를 실측 단위(mm)로 바꾸는 단계다.
- _deltasMm = _buildDeltas(_pointsMm) 는 점과 점 사이 변화량(dx, dy)을 계산한다.
- _currentZValues = _syncZList(...) 는 Z 리스트 길이를 포인트 개수에 맞춰 맞춘다. 부족하면 0.0 을 채운다.

9. 좌표 변환 함수: _canvasToImageClamped 와 _imageToCanvas

이 앱은 좌표계를 하나만 쓰지 않는다. 화면 표시 좌표, 프레임/이미지 좌표, 최종 mm 좌표가 있다. 그래서 좌표 변환 함수가 꼭 필요하다.

```
Offset _canvasToImageClamped(Offset p) {
 if (_frameRect == null) {
  return p;
 }
 final rect = _frameRect!;
 if (_userFrameMmSize != null || _imageSize == null) {
  final logicalSize = _userFramePxSize ?? Size(rect.width, rect.height);
  final localX = (p.dx - rect.left).clamp(0.0, rect.width);
  final localY = (p.dy - rect.top).clamp(0.0, rect.height);
  final x =
    rect.width == 0 ? 0.0 : localX / rect.width * logicalSize.width;
  final y =
    rect.height == 0 ? 0.0 : localY / rect.height * logicalSize.height;
  return Offset(x.toDouble(), y.toDouble());
 }
 final sx = _imageSize!.width / rect.width;
 final sy = _imageSize!.height / rect.height;
 final x = (p.dx - rect.left) * sx;
 final y = (p.dy - rect.top) * sy;
 final clampedX = x.clamp(0.0, _imageSize!.width);
 final clampedY = y.clamp(0.0, _imageSize!.height);
 return Offset(clampedX, clampedY);
}
```

- 화면에서 클릭한 위치는 전체 화면 기준이므로, 먼저 프레임의 left/top 을 빼서 프레임 내부 좌표로 바꾼다.
- clamp(최소, 최대) 는 값이 범위를 벗어나지 못하게 잘라 주는 함수다.
- 사용자 지정 프레임 크기를 쓰는 경우와 실제 이미지 크기를 쓰는 경우가 나뉜다.

- 결국 이 함수의 목표는 “캔버스에서 찍은 위치를 이미지/논리 좌표로 안전하게 바꿔 주는 것”이다.

10. 샘플링 함수: _resamplePolyline

손으로 그린 선은 점 간격이 일정하지 않다. 마우스를 빨리 움직이면 점이 듬성듬성하고, 천천히 움직이면 점이 빽빽해진다. 이 함수는 그 문제를 해결한다.

```
List<Offset> _resamplePolyline(List<Offset> input, double stepPx) {
  if (input.isEmpty) return <Offset>[];
  if (input.length == 1 || stepPx <= 0) return <Offset>[input.first];

  final result = <Offset>[input.first];
  var carry = 0.0;

  for (var i = 1; i < input.length; i++) {
    var a = input[i - 1];
    final b = input[i];
    var seg = (b - a).distance;
    if (seg == 0) continue;

    while (carry + seg >= stepPx) {
      final t = (stepPx - carry) / seg;
      final nx = a.dx + t * (b.dx - a.dx);
      final ny = a.dy + t * (b.dy - a.dy);
      final p = Offset(nx, ny);
      result.add(p);
      a = p;
      seg = (b - a).distance;
      carry = 0.0;
    }
    carry += seg;
  }

  if (result.last != input.last) {
    result.add(input.last);
  }
  return result;
}
```

- 핵심 목표는 점 간격을 stepPx 로 일정하게 맞추는 것이다.
- carry 는 이전 선분에서 남은 길이를 기억하는 변수다. 그래서 선분이 여러 개여도 전체 경로 기준으로 일정 간격 샘플링이 가능하다.
- while (carry + seg >= stepPx) 가 참이면 현재 선분 안에서 새 샘플 점을 하나 찍을 수 있다는 뜻이다.
- t 는 선분 a→b 사이에서 몇 퍼센트 위치인지 나타내는 비율이다.
- 마지막 점(input.last)을 따로 한 번 더 넣는 이유는 선 끝이 빠지지 않게 하기 위해서다.

11. 전송 준비: _buildPayload

이제 점들을 서버에 보내기 좋은 형태로 바꾼다. 이 함수는 최종 JSON 뼈대를 만든다.

```
Map<String, dynamic> _buildPayload() {
  final allPoints = <List<dynamic>>[];

  if (_strokes.isEmpty) {
    final zList = _syncZList(_currentZValues, _pointsMm.length);
    for (var i = 0; i < _pointsMm.length; i++) {
      final p = _pointsMm[i];
      final z = zList[i];
      allPoints.add(<double>[
        double.parse(p.dx.toStringAsFixed(2)),
        double.parse(p.dy.toStringAsFixed(2)),
        double.parse(z.toStringAsFixed(2)),
      ]);
    }
    return <String, dynamic>{'points': allPoints};
  }

  for (final stroke in _strokes) {
    final imagePoints =
        stroke.samples.map(_canvasToImageClamped).toList(growable: false);
    final pointsMm = _applyTransform(imagePoints, _mmPerPx);
    stroke.zValues = _syncZList(stroke.zValues, pointsMm.length);
    for (var i = 0; i < pointsMm.length; i++) {
      final p = pointsMm[i];
      final z = stroke.zValues[i];
      allPoints.add(<dynamic>[
        double.parse(p.dx.toStringAsFixed(2)),
        double.parse(p.dy.toStringAsFixed(2)),
        double.parse(z.toStringAsFixed(2)),
      ]);
    }
    allPoints.add(<dynamic>["000"]);
  }

  return <String, dynamic>{'points': allPoints};
}
```

- 최종 구조는 {"points": [...]} 형태다. 서버는 이 형식을 받아 해석한다.
- stroke 가 하나도 없으면 현재 임시 포인트(_pointsMm)만 보낸다.
- 각 점은 [x, y, z] 세 값으로 만들어진다.
- toStringAsFixed(2) 와 double.parse(...) 를 함께 쓰는 이유는 소수점 둘째 자리까지 깔끔하게 정리하기 위해서다.
- stroke 가 여러 개일 때는 모든 stroke 를 순서대로 이어 붙인다.
- 각 stroke 끝마다 ["000"] 마커를 넣는 이유는 서버가 “여기서 선 하나가 끝났다”고 구분할 수 있게 하기 위해서다.

12. 실제 전송: _sendJson

이 함수는 최종적으로 서버와 연결한 뒤 JSON 문자열을 보낸다.

```
Future<void> _sendJson() async {
  if (_strokes.isEmpty) {
    if ((_mode == DrawMode.line ||
        _mode == DrawMode.circle ||
        _mode == DrawMode.rect) &&
        (_p0 == null || _p1 == null)) {
      setState(() {
        _status = 'No line to send';
      });
      return;
    }
    if (_mode == DrawMode.pen && _rawPoints.length < 2) {
      setState(() {
        _status = 'No stroke to send';
      });
      return;
    }
  }
  try {
    await _ensureConnected();
    final payload = _buildPayload();
    final jsonText = jsonEncode(payload);
    _socket?.write('$jsonText\n');
    await _socket?.flush();
    setState(() {
      _status = 'Sent ${_payloadPointCount()} points';
    });
  } catch (error) {
    setState(() {
      _status = 'Send failed: $error';
    });
  }
}
```

- Future<void> 와 async 는 이 함수가 비동기 작업을 한다는 뜻이다. 네트워크 연결과 전송은 시간이 걸릴 수 있기 때문이다.
- 보낼 선이 없는 상황을 먼저 검사한다. 선이 없는데 전송 버튼을 눌러도 서버에 의미 없는 빈 데이터를 보내지 않게 하기 위해서다.
- await _ensureConnected() 는 먼저 TCP 소켓이 연결되어 있는지 확인하고, 아니면 연결을 시도한다.
- _buildPayload() 로 데이터 구조를 만들고, jsonEncode(payload) 로 실제 JSON 문자열로 바꾼다.
- _socket?.write(...) 의 ?. 는 소켓이 null 이 아닐 때만 write 하겠다는 뜻이다.
- 끝에 \n 을 붙이는 이유는 서버 쪽에서 한 줄 단위로 읽기 쉽게 하려는 경우가 많기 때문이다.
- 성공하면 상태창에 보낸 포인트 개수를 표시하고, 실패하면 예외 메시지를 상태창에 보여 준다.

13. 연결 함수: _ensureConnected

전송 전에 서버와 연결이 되어 있는지 확인하는 함수다. 이미 같은 주소로 연결되어 있으면 그대로 쓰고, 아니면 새로 연결한다.

```
Future<void> _ensureConnected() async {
  final host = _hostController.text.trim();
  final port = int.tryParse(_portController.text.trim());
  if (host.isEmpty || port == null) {
    setState(() {
      _status = 'Invalid host or port';
    });
    throw Exception('Invalid host or port');
  }

  if (_socket != null &&
      _connectedHost == host &&
      _connectedPort == port) {
    return;
  }
  if (_socket != null) {
    _socket?.destroy();
    _socket = null;
    _connectedHost = null;
    _connectedPort = null;
  }
  try {
    final sourceAddress = await _selectSourceAddress(host);
    _status = 'Connecting to $host:$port...';
    setState(() {});
    final socket = await Socket.connect(
      host,
      port,
      sourceAddress: sourceAddress,
      timeout: const Duration(seconds: 3),
    );
    ...
  } catch (error) {
    ...
    rethrow;
  }
}
```

- 사용자가 입력한 host, port 를 먼저 읽는다.
- int.tryParse(...) 는 포트 번호를 안전하게 숫자로 바꾸는 함수다. 실패하면 null 이 된다.
- 이미 같은 서버에 연결되어 있으면 return 으로 그냥 재사용한다.
- 다른 서버로 바꾸는 상황이면 기존 소켓을 끊고 관련 정보도 null 로 초기화한다.
- Socket.connect(...) 는 실제 TCP 연결을 시도하는 부분이다.
- timeout 을 3 초로 준 것은 너무 오래 멈춰 있지 않게 하기 위한 설정이다.

- 연결 실패 시 catch 로 들어가고, 상태 메시지에 실패 원인을 남긴다.

14. 마무리 정리: 각 함수의 역할을 한 문장으로

- _onPointerDown: 그리기를 시작하기 위한 상태를 준비한다.
- _onPointerMove: 움직이는 동안 현재 경로를 계속 갱신한다.
- _onPointerUp: 임시 경로를 완성된 stroke 로 확정한다.
- _rebuildPoints: 현재 상태를 실제 점 목록으로 다시 계산한다.
- _resamplePolyline: pen 선을 일정 간격 점들로 다시 샘플링한다.
- _canvasToImageClamped: 화면 좌표를 이미지/프레임 좌표로 안전하게 바꾼다.
- _buildPayload: 점들을 [x, y, z] JSON 구조로 정리한다.
- _sendJson: 서버 연결 후 JSON 문자열을 실제로 전송한다.
- _deleteSelectedOrLast: 선택된 선 또는 마지막 선을 삭제한다.
- _reset / _clearCurrentDrawing: 전체 상태 또는 현재 임시 상태를 정리한다.

15. 공부 팁

- 이 코드를 외울 필요는 없다. 먼저 “언제 실행되는 함수인지”를 기억하면 훨씬 이해하기 쉽다.
- 이벤트 함수는 시간 순서로 보면 된다: Down → Move → Up.
- 복잡해 보이면 변수 이름을 역할별로 묶어서 보자. 예: 입력 상태(_p0, _p1, _rawPoints), 저장된 선(_strokes), 변환 결과(_pointsMm), 연결 상태(_socket).
- 가장 중요한 핵심 3 개를 고르면 _onPointerDown, _rebuildPoints, _buildPayload 다. 시작, 계산, 전송 준비를 각각 담당하기 때문이다.